Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 14 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

Assignment Objective: Provision a clean target environment using Ansible, deploy the application or container, demonstrate idempotency, perform an application health check, and validate rollback/recovery to the previous stable release.

Technology Stack: Ansible, Docker, Docker Registry, Spring Boot, Apache Tomcat, MySQL, Maven, Git and GitHub.

1. 1. Overview

Week 14 extends the configuration management work completed in Week 13. Ansible was used to automate provisioning and reliability validation of the Grocery Inventory Dashboard.

The Week 14 implementation uses a dedicated Docker container and port so that the existing Week 12 deployment remains unaffected.

Component	Configuration
Automation tool	Ansible 2.21.4
Container platform	Docker 29.8.1
Container name	grocery-inventory-week14
Stable image	localhost:5001/grocery-inventory-dashboard:3
Application port	8084
Container port	8080
Database	MySQL
Registry	Local Docker Registry on port 5001
Target	Local Mac environment

Table 1: Week 14 Environment Configuration

The provisioning playbook was created at:

```
ansible/week14/provision.yml
```

The playbook verifies Docker availability, checks the existing container, determines whether deployment is required, deploys the stable image when necessary, performs an HTTP health check, and reports the final container state.

2. 2. Automated Provisioning

The Ansible playbook provisions the Week 14 reliability target using the stable Docker image:

```
localhost:5001/grocery-inventory-dashboard:3
```

The deployment uses the following configuration:

```
Container:
grocery-inventory-week14
```

```
Port mapping:
8084:8080

Application:
Spring Boot WAR deployed in Tomcat

Database:
jdbc:mysql://host.docker.internal:3306/grocery_inventory
```

The playbook first verifies that Docker is available. It then checks whether the Week 14 container already exists and retrieves its image and status.

If the container does not exist, uses a different image, or is not running, deployment is performed automatically.

The first provisioning execution successfully created the container and started the application.

The first run produced:

```
ok=11
changed=2
failed=0
```

The deployed container was running with the stable image and the application was accessible through port 8084.

3. 3. Application Health Check

After deployment, Ansible performs an HTTP health check against:

```
http://localhost:8084/automated-grocery-inventory-dashboard/items
```

The playbook waits for the application to start and retries the request until HTTP status 200 is received.

The successful health check confirmed that:

- the Docker container was running,
- the Tomcat application had started,
- the Spring Boot application was responding,
- the application could connect to the configured MySQL database,
- the inventory endpoint was accessible.

The final recovery validation also returned:

```
HTTP/1.1 200
Content-Type: text/html;charset=UTF-8
Content-Language: en-US
```

4. 4. Idempotency Validation

Idempotency was validated by executing the same Ansible playbook a second time while the correct Week 14 container was already running.

The playbook detected:

```
Existing container found: YES
Existing image:
localhost:5001/grocery-inventory-dashboard:3

Existing status:
running

Deployment required: False
```

Because the desired state was already satisfied, the remove, pull, and deploy operations were skipped.

The second execution produced:

```
ok=13
changed=0
unreachable=0
failed=0
skipped=4
```

The value `changed=0` demonstrates that rerunning the automation did not unnecessarily recreate or modify the already-correct deployment.

This provides evidence that the provisioning automation is repeatable and maintains the desired target state.

5. 5. Rollback and Recovery Demonstration

A controlled deployment failure was introduced to validate recovery.

The stable application uses:

```
8084 -> 8080
```

For the failure scenario, the container was intentionally recreated with an incorrect mapping:

```
8084 -> 9999
```

The application endpoint could no longer be reached successfully, producing a connection failure.

The failed container was removed and the previous stable configuration was restored using:

```
Image:
localhost:5001/grocery-inventory-dashboard:3

Correct port mapping:
8084:8080
```

The recovered deployment was then checked using the application endpoint. The health check returned HTTP 200.

The final Docker inspection confirmed:

```
Image=localhost:5001/grocery-inventory-dashboard:3
Status=running
HostPort=8084
ContainerPort=8080
```

Thus, the controlled failed deployment was recovered to the known stable release configuration.

6. 6. Execution Evidence

The following execution logs were committed to the project repository:

```
ansible/week14/provision.yml
ansible/week14/provision-first-run.log
ansible/week14/provision-idempotency.log
ansible/week14/rollback-recovery.log
```

The logs provide persistent evidence of the provisioning, idempotency and rollback-/recovery activities.

The rollback/recovery log records the final stable state:

```
Image=localhost:5001/grocery-inventory-dashboard:3
Status=running

HTTP/1.1 200

Ports:
8080/tcp -> HostPort 8084
```

7. 7. Git and Release Baseline

The Week 14 implementation was developed on:

```
feature/ansible-provisioning-reliability
```

The implementation was committed using:

291efb5

Add Ansible provisioning and reliability validation

A GitHub Pull Request was created and merged into the `develop` branch.

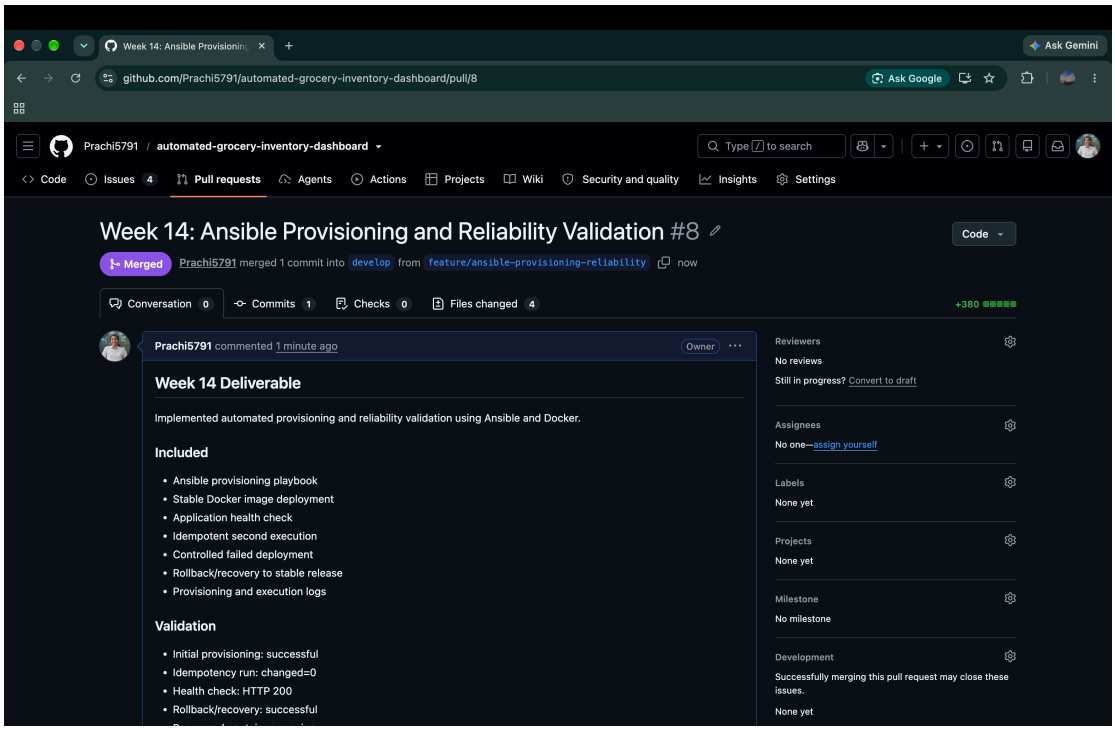


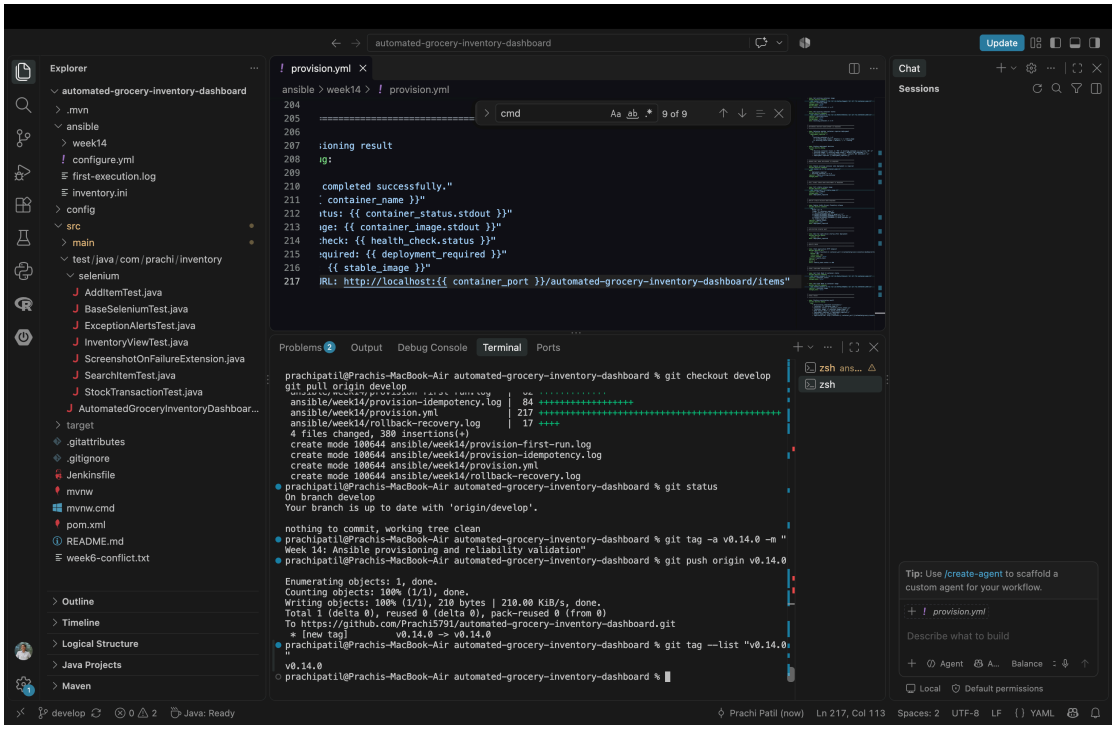
Figure 1: Week 14 Pull Request Merged into develop

After merging, the local `develop` branch was synchronized with `origin/develop`. The working tree was clean.

The Week 14 baseline was then tagged as:

v0.14.0

and pushed to the GitHub repository.


automated-grocery-inventory-dashboard
Update
Explorer
provision.yml
Chat
Sessions
ansible > week14 > provision.yml
cmd
9 of 9
Problems 2
Output
Debug Console
Terminal
Ports
prachipatil@Prachis-MacBook-Air automated-grocery-inventory-dashboard % git checkout develop
git pull origin develop
ansible/week14/provision-idempotency.log | 84 ++++++++++++++++++
ansible/week14/provision.yml | 217 ++++++++++++++++++